BCSE-0611 Application & Data Security

Dr. Arvind Prasad
Department of CEA
GLA University, Mathura

Mobile Security

Introduction to Mobile Security

Mobile Security refers to protecting smartphones, tablets, and other portable devices from threats such as data theft, malware, unauthorized access, and network attacks.

Why It Matters:

- Mobile devices store **personal, financial, and corporate data**.
- High risk of **loss or theft**.
- Frequent **internet connectivity** → exposure to phishing, malware, and network attacks.
- Use of **third-party apps** can introduce vulnerabilities.

Common Threats:

- **Phishing Attacks** – fake apps or links stealing credentials.
- **Malware & Spyware** – malicious code stealing or tracking user data.
- **Data Leakage** – via insecure apps or permissions.
- **Network Attacks** – man-in-the-middle, Wi-Fi spoofing.
- **Device Theft** – physical compromise of data.

GLA UNIVERSITY MATHURA

Why Focus on Android Security

Why Android?

- Over **70% of mobile devices** run Android.
- **Open-source nature** makes it flexible but more exposed to malicious modifications.
- Allows installation from **unverified sources (APK sideloading)**.
- Highly **customizable**, leading to fragmentation and inconsistent security patches.

Key Security Goals:

- **Confidentiality:** Protect user data.
- **Integrity:** Prevent unauthorized modification.
- **Availability:** Ensure secure and stable operation.
- **Authenticity:** Verify apps and sources.
- **Isolation:** Contain the impact of malicious apps.

GLA UNIVERSITY MATHURA

Android Architecture Overview

Five Key Layers:

1. Applications
2. Application Framework
3. Android Runtime
4. Platform Libraries
5. Linux Kernel

GLA UNIVERSITY MATHURA

Android Security Overview

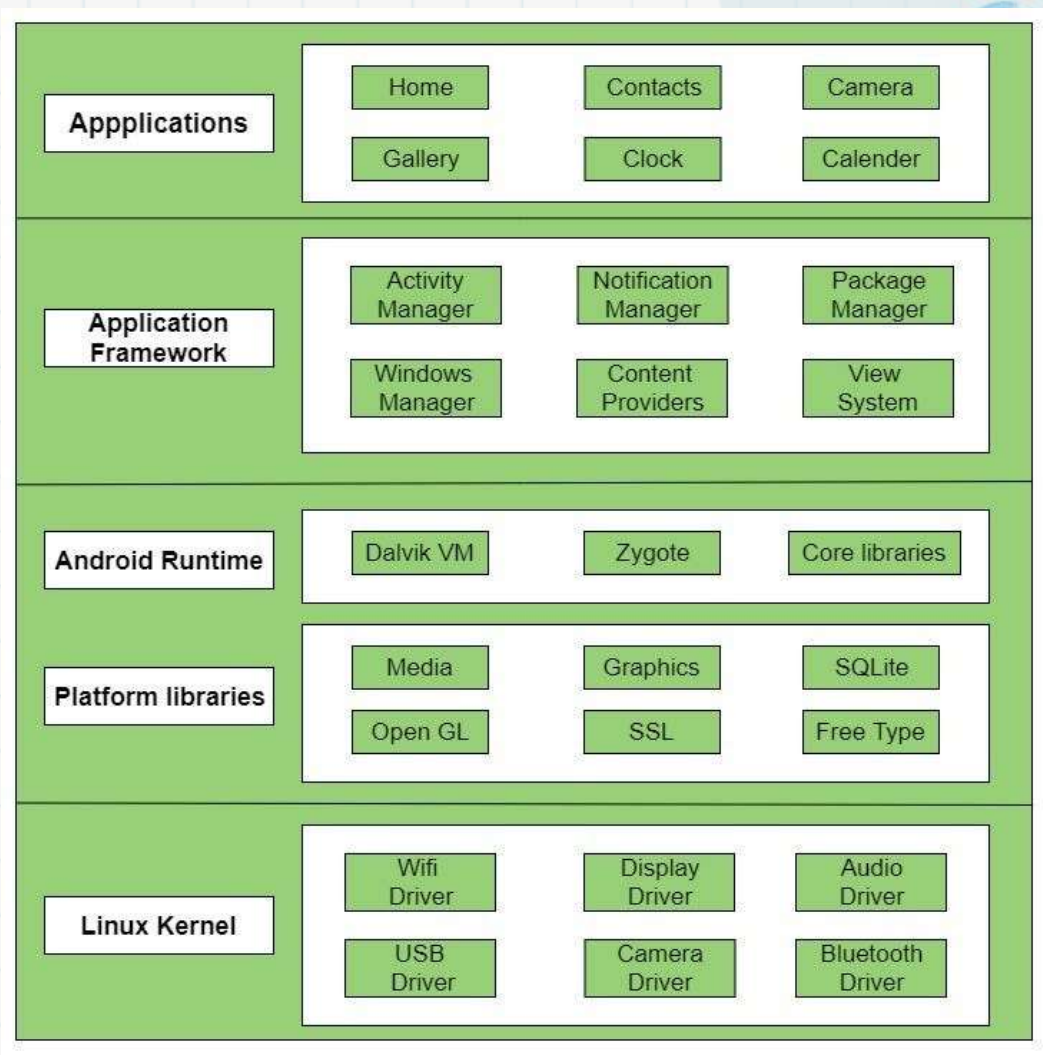
Android Security uses a **multi-layered defense-in-depth model** — meaning each layer of the architecture contributes to overall protection.

Main Security Mechanisms:

- **Sandboxing:** Isolates apps from each other.
- **Permissions System:** User consent for sensitive operations.
- **Google Play Protect:** Scans apps for malicious behavior.
- **SELinux:** Enforces strict access control at the kernel level.
- **Regular Security Patches:** Fix known vulnerabilities.

GLA UNIVERSITY MATHURA

Architecture



Applications Layer

Description:

- The topmost layer where user and system apps run (Home, Camera, Gallery, Clock, etc.).
- Apps communicate through defined interfaces — no direct hardware or kernel access.

Security Role:

- **Sandboxing:** Each app runs in an isolated environment.
- **Permissions:** Apps request user approval to access resources.
- **App Signing:** Ensures integrity and authenticity of applications.

GLA UNIVERSITY MATHURA

Application Framework Layer

Description:

- Provides reusable APIs and system services for app developers.
- Components:
  - Activity Manager
  - Notification Manager
  - Content Providers
  - Package Manager
  - Window Manager
  - View System

Security Role:

- Controls **data sharing** through Content Providers.
- Enforces **permissions and policy checks** before granting resource access.
- Manages secure **inter-process communication (IPC)**.

GLA UNIVERSITY MATHURA

Android Runtime Layer

Description:

- Core part of Android responsible for executing applications.
- Contains:
  - **ART (Android Runtime)** — executes app bytecode.
  - **Zygote** — preloads system classes for faster app launch.
  - **Core Libraries** — provide Java-based system functionalities.

Security Role:

- **Runtime Permissions:** Apps request access dynamically.
- **Memory Management:** Prevents buffer overflows and memory corruption.
- **Code Verification:** Ensures apps run safely within a managed environment.

GLA UNIVERSITY MATHURA

Platform Libraries Layer

Description:

- Includes essential C/C++ libraries for system operations (Media, Graphics, SQLite, SSL, FreeType, OpenGL).

Security Role:

- **SSL/TLS:** Secure network communication (HTTPS).
- **SQLite:** Manages secure, isolated data storage.
- **Library Hardening:** Prevents exploits in native code.

GLA UNIVERSITY MATHURA

Linux Kernel Layer

Description:

- The foundation of Android OS; manages process, memory, and hardware.
- Includes hardware drivers (Wi-Fi, Camera, Bluetooth, Audio, USB).

Security Role:

- **Process Isolation:** Each app has a unique user ID (UID).
- **SELinux (Security-Enhanced Linux):** Enforces mandatory access control.
- **Hardware Abstraction:** Prevents apps from directly accessing device drivers.
- **Secure Boot & Verified Boot:** Ensure OS integrity from startup.

GLA UNIVERSITY MATHURA

Summary — Security by Design

Layer	Key Security Features
Applications	Sandboxing, App Signing, Permissions
Framework	Access Control, Data Protection
Runtime	Code Verification, Memory Safety
Libraries	Secure Communication, Encrypted Storage
Kernel	SELinux, Process Isolation, Hardware Protection

GLA UNIVERSITY MATHURA

Linux Kernel

- The **core** of Android OS — built on the **Linux kernel**.
- Handles **low-level system services like process management, memory management, power management, device drivers, and security**.
- Provides **hardware abstraction** — meaning apps don't access hardware directly (e.g., camera, Wi-Fi, Bluetooth).
- Uses **process isolation** and **user-based permissions (UIDs)** to sandbox apps.
- Implements **SELinux (Security-Enhanced Linux)** for enforcing strict access control. Defines strict rules about what each app or system component is allowed to do. Even if an app is hacked, SELinux prevents it from accessing things it shouldn't.

GLA UNIVERSITY MATHURA

Libraries

- A set of **C/C++ libraries** that provide key functionalities to higher layers.
- These libraries are used by the Android Runtime and Application Framework.
- Examples:
  - **OpenGL ES:** Graphics rendering
  - **SQLite:** Database storage
  - **WebKit:** Browser engine
  - **SSL:** Secure network communication
  - **Media Framework:** Audio/video playback and recording

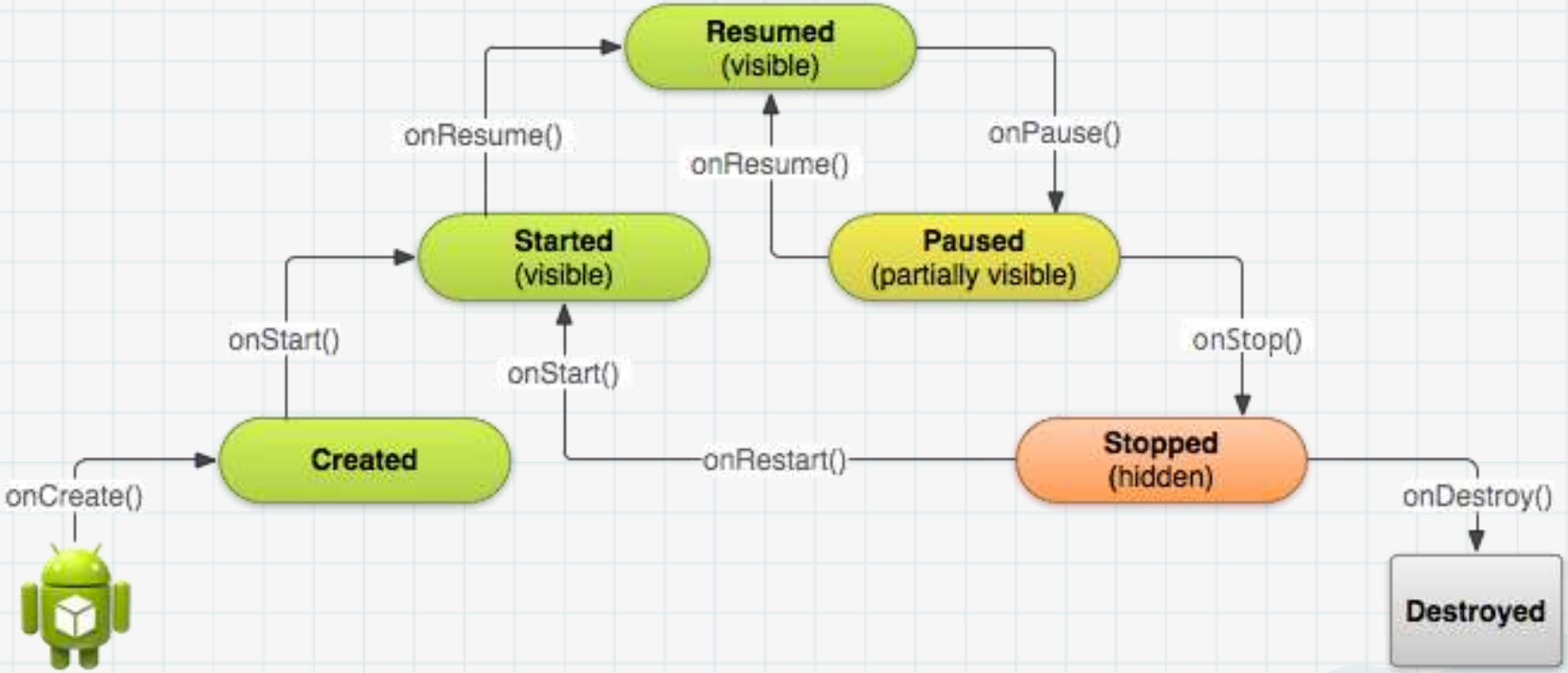
GLA UNIVERSITY MATHURA

Android Runtime (ART)

- This layer runs and manages **Android applications**.
- **ART (Android Runtime)** replaced the older **Dalvik VM**.
- Each app runs in its **own process and sandboxed environment**.
- ART compiles app code **Ahead-of-Time (AOT)** into native machine code, improving performance.
- Includes **core Java libraries** that provide basic Android functionality.
- Handles **memory management**, **garbage collection**, and **runtime permissions**.

GLA UNIVERSITY MATHURA

Activity Life Cycle



GLA UNIVERSITY MATHURA

Android Activity Lifecycle

onCreate()

→ Called when the activity is first created.

→ Initialize the UI, bind views, set up essential components, restore state.

onStart()

→ Activity becomes visible to the user (but not yet in the foreground).

→ Prepare UI-related updates.

onResume()

→ Activity moves to the foreground and becomes interactive.

→ Start animations, listeners, or any processes that need to run while visible.

onPause()

→ Another activity is partially covering this one.

→ Pause ongoing tasks, animations, sensors; commit unsaved changes.

GLA UNIVERSITY MATHURA

Android Activity Lifecycle

onStop()

→ Activity is completely hidden.

→ Release heavy resources, unregister receivers, save persistent data.

onRestart()

→ Called when the activity transitions from *stopped* back to *started*.

→ Re-initialize components released in onStop().

onDestroy()

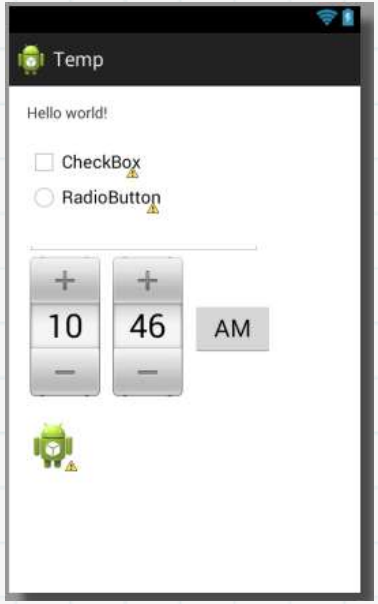
→ Final cleanup before the activity is destroyed.

→ Release all remaining resources.

GLA UNIVERSITY MATHURA

Views

- A building block for UI components
- Responsible for drawing and event handling
- Each View has an id
- findViewById(int)



Text box, check box, radio button, time picker, and image view

XML

- Eclipse IDE
- Uses XML files to set up the mobile application

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context="com.example.temp.MainActivity$PlaceholderFragment" >

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/hello_world" />

</RelativeLayout>
```

Android Layout File

GLA UNIVERSITY MATHURA

XML - Android Manifest

- Contains properties of the application
- Permissions, SDK, Icon, Activities

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.kopczyk.hos"
    android:versionCode="1"
    android:versionName="1.0" >

    <uses-permission android:name="android.permission.VIBRATE" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
    <uses-sdk
        android:minSdkVersion="11"
        android:targetSdkVersion="18" />

    <application
        android:allowBackup="true"
        android:icon="@drawable/hos_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme"
        android:hardwareAccelerated="true"  >

        <activity
            android:name="com.kopczyk.hos.ActivityHos"
            android:label="@string/app_name"
            android:screenOrientation="landscape" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Android Manifest File

XML - Layout

- Defines the layout of the Activity
- Set View id values
- Could achieve the same results through java code

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context="com.example.temp.MainActivity$PlaceholderFragment" >

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/hello_world" />

</RelativeLayout>
```

Android Layout File



Input Events

- OnClickListener
- OnLongClickListener
- OnFocusChangeListener
- OnKeyListener
- OnTouchListener
- OnCreateContextMenuListener

GLA UNIVERSITY MATHURA

Permission

- Allows developers to use security features
- Provides additional capabilities to consumers that otherwise would be impossible

"A central design point of the Android security architecture is that no application, by default, has permission to perform any operations that would adversely impact other applications, the operating system, or the user"

GLA UNIVERSITY MATHURA

Permission

- When an Application is installed the consumer must accept the permissions requested by an application
- Permissions are defined in the Manifest file

GLA UNIVERSITY MATHURA

Permission

- Facebook Messenger
  - Identity
  - Contacts/Calendar
  - Location
  - SMS
  - Phone
  - Photos/Media/Files
  - Camera/Microphone
  - Wi-Fi Connection Information
  - Device ID & Call Information
- Angry Birds
  - •Identity
  - •In-app purchases
  - •Location
  - •Photos/Media/Files
  - •Wi-Fi Connection Information
  - •Device ID & Call Information

GLA UNIVERSITY MATHURA

Rooting (Android)

- Rooting is the process of **gaining privileged (root) access** to an Android device's operating system.
  It allows the user (or attacker) to modify system files, install unauthorized apps, and bypass built-in security controls.
- **Pros (for advanced users):**
  - Full control over device and OS.
  - Custom ROMs, performance tuning, deep customization.
- **Security Risks:**
  - Removes Android's built-in sandboxing and app isolation.
  - Increases risk of **malware**, **data theft**, and **system compromise**.
  - Voids warranty and disables secure apps (e.g., banking, UPI).

Jailbreaking (iOS)

Jailbreaking is the **iOS equivalent of rooting** — it removes Apple's software restrictions to gain access to the iPhone's file system and install non-App Store apps.

Pros:

- More customization, third-party app installation.

Security Risks:

- Disables iOS security layers (code signing, sandboxing).
- Makes device vulnerable to **malware** and **remote exploits**.
- Sensitive apps (like banking) often refuse to run on jailbroken devices.

GLA UNIVERSITY MATHURA

Rooting vs. Jailbreaking:

Aspect	Rooting (Android)	Jailbreaking (iOS)
Purpose	Gain root access for customization, removing OEM limits.	Bypass Apple's restrictions to install non-App Store apps.
OS	Android (Linux-based)	iOS (Apple)
Method	Exploits OS vulnerabilities to unlock root access.	Uses iOS exploits to disable Apple's code-signing checks.
Security Impact	Breaks app sandboxing, allows malware full system access.	Disables iOS security layers (sandboxing, app signing).
Official Support	Not allowed by Google; voids warranty.	Not allowed by Apple; voids warranty.

GLA UNIVERSITY MATHURA

Preventive Measures:

- Avoid unauthorized system modifications or privilege escalation tools.
- Use only official app marketplaces and verified updates.
- Apply security patches regularly.
- Enable device integrity checks (e.g., **verified boot, integrity attestation**).
- Use enterprise policies or mobile device management (MDM) to block altered devices.
- Educate users about the risks of unrestricted system access.

GLA UNIVERSITY MATHURA

Mobile Application Security Vulnerabilities — OWASP Top 10

The **OWASP (Open Web Application Security Project)** Mobile Top 10 lists the most critical security risks found in **mobile applications**.
It helps developers, testers, and security analysts identify and mitigate common weaknesses in Android and iOS apps.

GLA UNIVERSITY MATHURA

M1 – Improper Platform Usage

Occurs when a mobile app **misuses platform-specific features or permissions**, or fails to use security controls properly.

Examples:

- Misusing **Android intents or iOS URL schemes**.
- Failing to enforce permissions (e.g., camera, location).
- Ignoring platform security features like Keychain or Keystore.

Mitigation:

- Follow **official platform security guidelines**.
- Use **Android Permissions Model** and **KeyStore API** properly.
- Disable **exported components** unless required.

GLA UNIVERSITY MATHURA

M2 – Insecure Data Storage

Sensitive data is stored **unencrypted** or insecurely on the device's file system, shared preferences, or databases.

Examples:

- Storing passwords, tokens, or personal info in plain text.
- Saving sensitive logs or caches that can be read by other apps.

Mitigation:

- Use **encryption** for local data.
- Store secrets only in **Secure Enclave/KeyStore**.
- Avoid writing sensitive data to logs or external storage.

GLA UNIVERSITY MATHURA

M3 – Insecure Communication

Occurs when the app transmits sensitive data over **unsecured or poorly configured network connections**.

Examples:

- Using **HTTP instead of HTTPS**.
- No SSL/TLS certificate validation.
- Allowing weak ciphers or old protocols.

Mitigation:

- Always use **HTTPS/TLS 1.2+**.
- Implement **certificate pinning**.
- Validate SSL certificates strictly.
- Never transmit sensitive data in URLs or GET parameters.

GLA UNIVERSITY MATHURA

M4 – Insecure Authentication

Weak or flawed authentication mechanisms that allow attackers to bypass login or impersonate users.

Examples:

- Relying only on device identifiers (IMEI, MAC).
- Poor session management (no expiry, predictable tokens).
- No MFA (Multi-Factor Authentication).

Mitigation:

- Use strong **multi-factor authentication (MFA)**.
- Implement secure **session tokens** and **expiry mechanisms**.
- Avoid storing credentials locally.
- Enforce server-side authentication checks.

GLA UNIVERSITY MATHURA

M5 – Insufficient Cryptography

Using **weak, outdated, or incorrect cryptographic algorithms** to protect data.

Examples:

- Using **MD5/SHA-1** instead of modern algorithms.
- Hardcoding cryptographic keys in source code.
- Poor random number generation.

Mitigation:

- Use proven, up-to-date algorithms (**AES-256, SHA-256**).
- Use **KeyStore/Keychain** for key management.
- Never hardcode keys or credentials in the app.
- Use hardware-backed cryptography if available.

GLA UNIVERSITY MATHURA

M6 – Insecure Authorization

Flawed implementation of access control that allows users to **access data or functionality beyond their privileges**.

Examples:

- A user modifying an API request to access another user's records.
- Missing server-side role validation.

Mitigation:

- Enforce **server-side authorization checks**.
- Use **Role-Based Access Control (RBAC)**.
- Validate every request against user privileges.

GLA UNIVERSITY MATHURA

M7 – Client Code Quality Issues

Poor coding practices such as unhandled exceptions, insecure logic, or buffer overflows in the client-side code.

Examples:

- Lack of input validation in native code (C/C++).
- Debug code left in production.

Mitigation:

- Follow **secure coding standards**.
- Perform **static and dynamic code analysis (SAST/DAST)**.
- Use safe functions and proper exception handling.

GLA UNIVERSITY MATHURA

M8 – Code Tampering

Attackers modify or repackage the app's code (APK/IPA) to add malicious functionality.

Examples:

- Reverse-engineering and injecting malware into a legitimate app.
- Removing license verification or security checks.

Mitigation:

- Implement **code signing and integrity verification**.
- Use **runtime integrity checks** and **tamper detection**.
- Obfuscate code to make reverse engineering harder.

GLA UNIVERSITY MATHURA

M9 – Reverse Engineering

Attackers decompile or analyze the app's binary to extract sensitive information or understand its logic.

Examples:

- Using tools like **APKTool, JADX, or IDA Pro** to view source code.
- Extracting API keys or encryption logic.

Mitigation:

- Use **code obfuscation** (e.g., ProGuard, R8).
- Encrypt critical strings and resources.
- Store secrets on secure servers, not in the app.

GLA UNIVERSITY MATHURA

M10 – Extraneous Functionality

Extraneous functionality are **functions or secrets hidden inside the app** (like test code or debug endpoints) that expose internal functions or data.

Examples:

- Debug logs, test APIs, admin panels left in production.
- Hardcoded backdoor credentials.

Mitigation:

- Remove **debug and test code** before release.
- Conduct **security reviews** and **penetration testing**.
- Enforce **secure deployment practices**.

GLA UNIVERSITY MATHURA

Summary Table

ID	Vulnerability	Core Risk	Key Mitigation
M1	Improper Platform Usage	Misuse of OS features	Follow platform security APIs
M2	Insecure Data Storage	Data leakage	Encrypt local data
M3	Insecure Communication	MITM attacks	Enforce HTTPS & TLS
M4	Insecure Authentication	Account takeover	Use MFA & secure sessions
M5	Insufficient Cryptography	Weak encryption	Use modern crypto algorithms
M6	Insecure Authorization	Privilege escalation	Server-side access control
M7	Client Code Quality	Memory corruption	Secure coding & code review
M8	Code Tampering	Repackaged malware	Code signing & integrity check
M9	Reverse Engineering	Logic exposure	Obfuscation & encryption
M10	Extraneous Functionality	Hidden backdoors	Remove debug/test code

GLA UNIVERSITY MATHURA

Mobile APK Penetration Testing

What is Mobile APK Penetration Testing?

A simulated cyber attack against an Android Application Package (APK) conducted by **authorized security professionals** to identify and exploit security vulnerabilities.

Key Mindset

Authorized & Legal: Always obtain explicit permission before testing

Simulates Real Attackers: Use same tools/techniques as malicious hackers

Goal-Oriented: Focus on business impact and risk

Comprehensive: Test both client (APK) AND server components

GLA UNIVERSITY MATHURA

Primary Legal and Ethical Risks during Mobile APK Penetration Testing

- Testing without written authorization may violate laws (e.g., Computer Misuse or Cybercrime Acts).
- Accessing live user data breaches privacy and data protection regulations (e.g., GDPR).
- Risk of unauthorized data exposure or system disruption during testing.

Recommended Safeguards during Testing during Mobile APK Penetration Testing

- Use a staging or test environment instead of production systems.
- Mask any sensitive or personal data encountered.
- Maintain detailed logs and communication with the client to ensure transparency and accountability.

GLA UNIVERSITY MATHURA

Primary Legal and Ethical Risks while Mobile APK Penetration Testing

- Testing without written authorization may violate laws (e.g., Computer Misuse or Cybercrime Acts).
- Accessing live user data breaches privacy and data protection regulations (e.g., GDPR).
- Risk of unauthorized data exposure or system disruption during testing.

Testing Environment & Tooling

Essential Setup

1. **Android Studio & SDK** – Official tools, emulators, ADB
2. **Rooted Device/Emulator** – Critical for deep analysis
   a. Google AVD (Android Virtual Device)
3. **Intercepting Proxy** – Burp Suite or OWASP ZAP

Core Testing Tools

Static Analysis (White-Box)

1. **Jadx–** #1 tool for decompiling to Java/Kotlin
2. **APKTool** – Disassembles to Smali code, allows repackaging
3. **MobSF** – Automated static analysis framework

Dynamic Analysis (Black-Box)

1. **Frida** – Dynamic instrumentation toolkit
2. **Objection** – Runtime exploration (built on Frida)

Advanced Reverse Engineering

1. **Ghidra/IDA Pro** – For analyzing native libraries (.so files)

GLA UNIVERSITY MATHURA

Testing Process

A.Reconnaissance (pre-test)

B. Static Analysis (offline)

C. Dynamic / Runtime Analysis

D. Tampering & Repackaging

E. Post-Exploitation Checks

A. Reconnaissance (pre-test)

Reconnaissance in cybersecurity is the initial phase of a cyberattack where attackers gather information about a target to identify vulnerabilities and plan an attack. This can be done through **passive methods**, which collect data from public sources without direct interaction, or **active methods**, which involve probing the target's systems directly. The goal is to build a comprehensive profile of the target's network, systems, and potential weaknesses to find the best entry point.

Identify app version, target SDK, permissions.
Get APK: from Play Store (via downloader), from device adb pull /data/app/... or provided by client.
Read app description and functionality to prioritize sensitive flows (auth, payments, storage).

GLA UNIVERSITY MATHURA

B. Static Analysis (offline)

Unpack APK

- apktool d app.apk -o app_src (resources + smali)
- jadx-gui app.apk (readable Java source)

Manifest analysis

- AndroidManifest.xml: exported components, permissions, intent-filters, allowBackup, debuggable, usesCleartextTraffic.
- Look for android:exported="true", android:debuggable="true", android:allowBackup="true".

Code inspection

- Search for API keys, hardcoded secrets, credentials, weak crypto, insecure URL endpoints.
- Check use of WebView with addJavascriptInterface, loadUrl patterns.
- Check use of insecure crypto (MD5, SHA1, ECB mode, hardcoded keys).

GLA UNIVERSITY MATHURA

B. Static Analysis (offline)

Resources & assets

- Inspect res/, assets/ for config files, secrets, certificates, .keystore files.

Native libs

- Disassemble .so with Ghidra for logic hidden in native code.

Third-party libs

- Identify outdated libraries with known CVEs.

Build config

- Look for ProGuard/R8 obfuscation presence. If not present, easier to reverse.

GLA UNIVERSITY MATHURA

C. Dynamic / Runtime Analysis

Dynamic / Runtime Analysis

Install on test device

- adb install -r app.apk

Run and observe logs

- adb logcat — look for leaks, stack traces, tokens printed.

Network interception

- Configure device/emulator to proxy through Burp.
- Install Burp's CA on device (user or system CA depending on Android API and cert-pinning).
- Test API calls, observe sensitive data in requests/responses (tokens, PII).

Certificate pinning

- If pinning prevents interception, test bypass:
  - Patch app (replace okhttp/ssl logic) or use Frida scripts (e.g., frida-ssl-pinning-bypass), or run on rooted device with Xposed module JustTrustMe/SSLUnpinning, or patch Network Security Config.

GLA UNIVERSITY MATHURA

C. Dynamic / Runtime Analysis

Runtime hooking

- Use Frida to hook functions (e.g., java.crypto, KeyStore calls, WebView methods, auth flow).
- frida -U -f your.package.name -l script.js --no-pause

Input fuzzing & IPC

- Send crafted Intents/Broadcasts to exported components to test parameter handling and privilege escalation.
- Example: adb shell am start -n com.example/.SensitiveActivity --es amount 100000

Local storage checks

- Inspect app storage: /data/data/<pkg>/ (requires root) for databases, shared_prefs, files.
- Check for plaintext tokens, passwords, PII.

Behaviour under rooting/jailbreak

- Test whether app detects rooted devices and how it behaves; check for logic bypasses.

GLA UNIVERSITY MATHURA

D. Tampering & Repackaging

Repack flow:

- apktool d app.apk
- Modify payload (e.g., add malicious activity or bypass authentication)
- apktool b app_src → output unsigned APK
- Sign APK: apksigner sign --ks mykeystore.jks app_unsigned.apk
- Install and run; observe behavior

Purpose: test anti-tamper, integrity checks, signature checks.

GLA UNIVERSITY MATHURA

E. Post-Exploitation Checks

If you can perform actions (change PIN, access data), evaluate impact and persistence.

Try to exfiltrate sensitive data, persist background services, or escalate privileges.

GLA UNIVERSITY MATHURA

Common Vulnerabilities & How to Test / Exploit

1. Exported components
2. Insecure Data Storage
3. Insecure Communication / No TLS or wrong TLS
4. Hardcoded secrets
5. Insecure WebView usage
6. Insufficient authentication and session management
7. Signature / Integrity checks absent
8. Reverse-engineering / Lack of obfuscation
9. Insecure inter-app communication (Intents & URL Schemes)
10. Insecure use of cryptography

TLS (Transport Layer Security)

What it is: A cryptographic protocol designed to provide secure communication over a computer network.

Primary Goal: To ensure **privacy**, **data integrity**, and **authentication** between two communicating applications (e.g., a web browser and a server).

Key Features & Benefits

- **Encryption:** Scrambles data to prevent eavesdropping (confidentiality). Uses a combination of **asymmetric encryption** (for the handshake) and faster **symmetric encryption** (for the data transfer).
- **Authentication:** Uses digital certificates to verify the identity of the server (and optionally the client). This prevents man-in-the-middle attacks.
- **Data Integrity:** Uses Message Authentication Codes (MACs) to ensure that data sent between client and server is not tampered with or altered in transit.

GLA UNIVERSITY MATHURA

Intercepting TLS Traffic

TLS is a protocol for setting up and using a cryptographically secured connection between two computer systems, a client and a server.

TLS (Transport Layer Security) interception is the process of **decrypting**, **inspecting**, and then **re-encrypting** secure HTTPS traffic as it passes through a middlebox. This middlebox acts as a "Man-in-the-Middle" (MitM), breaking the end-to-end encryption between a client (e.g., a user's browser) and a server (e.g., google.com).

TLS interception involves capturing obtaining the *plain text* of an encrypted transport (usually "HTTPS" using *TLS (Transport Layer Security)* previously/commonly known as *SSL (Secure Sockets Layer)*) either by collecting the encrypted stream and decoding it at a later stage, or more conventionally by posing as the sender (to the receiver) and receiver (to the sender). Security researchers refer to this as a **man-in-the-middle attack (MITM)** when applied without user knowledge. Vendors that supply equipment for this purpose often refer to the capacity as **SSL inspection**.

GLA UNIVERSITY MATHURA

The Static Analysis with JADx

JADx is a powerful, open-source tool that **decompiles** Android DEX files (the compiled bytecode) back into a very readable, high-level Java source code. It's often more accurate and produces cleaner output than many other decompilers.

Step 0: Preparation & Safety First!

- **Isolate the Environment:** Perform analysis in a virtual machine (e.g., VirtualBox, VMware) that is not connected to your personal or corporate network. Malware can attempt to escape or infect the host system.
- **Acquire the Sample:** The suspicious APK file will be your sample (e.g., suspicious_app.apk).
- **Have JADx Ready:** You can use the command-line tool (jadx) or the GUI (jadx-gui). The GUI is excellent for exploratory analysis.

GLA UNIVERSITY MATHURA

Cont….

Step 1: Initial Reconnaissance with JADx

Open the APK file with jadx-gui. The left panel will show the project structure.

Examine the AndroidManifest.xml:

This is the app's configuration file. It tells you everything the app *says* it needs to do.

- **Package Name:** Note the unique identifier (e.g., com.example.fake.cleaner). Is it trying to impersonate a legitimate app?
- **Permissions**
- **Components (Activities, Services, Receivers, Providers):**
  - Look for exported components (android:exported="true"), which can be triggered by other apps. This is a common privilege escalation vector.
  - Note the main activity (the app's entry point).

Scan the resources/ Folder:

- Look in res/values/strings.xml for hardcoded URLs, IP addresses, passwords, or encryption keys.
- Check other resource files for any embedded configuration data.

GLA UNIVERSITY MATHURA

Cont….

Step 2: Code Analysis - The Hunt for Malicious Logic

This is the core of the analysis. Use the search function (Ctrl+F) extensively.

Search for Indicators of Compromise (IoCs):

- **URLs & IPs:** Search for "http://", "https://", and IP address patterns. These are often Command & Control (C&C) servers.
- **File Paths:** Look for references to sensitive files (/data/data/, /sdcard/).
- **Suspicious Strings:** Search for keywords like:
  - "su" (Superuser – rooting)
  - "runtime.exec", "ProcessBuilder" (Executing shell commands)
  - "getDeviceId", "getSubscriberId" (Hardware identifiers)
  - "crypto", "AES", "DES" (Encryption routines, might be for data exfiltration)
  - "banking", "paypal", etc. (Targeting specific apps)

Cont….

Analyze the Entry Points:

- Start from the MainActivity (found via the AndroidManifest.xml). Trace the code from onCreate().
- Examine other components:
  - **Services:** Look for long-running background tasks (e.g., onStartCommand). Malware uses these for persistence.
  - **BroadcastReceivers:** See what system events (e.g., BOOT_COMPLETED) trigger the app's code. BOOT_COMPLETED is a classic persistence mechanism.
  - **ContentProviders:** Check for data leakage through improperly secured providers.

GLA UNIVERSITY MATHURA

Cont….

Understand the Malware's "Kill Chain":

- **Dropper/Payload:** Is the app downloading and executing another APK? Look for code that uses PackageManager to install apps or DexClassLoader to load external code.
- **Data Theft:** Is it reading SMS, contacts, or files? Search for ContentResolver queries for content://sms/ or content://contacts/.
- **Evasion:** Does it check for **emulators, rooted devices, or debugging**? Look for checks against Build properties, **isDebuggerConnected**(), or the presence of specific files like /system/bin/su.
- **Communication:** How does it talk to the C&C server? Analyze the network code, often found in classes with names like NetworkUtils, HttpRequest, or CommandHandler.

GLA UNIVERSITY MATHURA

Android Malware analysis dynamic analysis (Sandbox)

Dynamic analysis involves running the malware in an isolated, monitored environment to observe its:

- **Network communications**
- **File system operations**
- **API calls**
- **System interactions**
- **User interface behavior**

Why Dynamic Analysis for Android Malware?

Static analysis can be defeated by:

- **Obfuscation** (renaming, encryption, packing)
- **Native code** (C/C++ libraries)
- **Runtime loading** (DexClassLoader)
- **Conditional logic** (only activates in specific circumstances)

Dynamic analysis reveals what the malware **actually does** when executed.

Dynamic Analysis Setup & Sandbox Environments

Step 0: Critical Safety Precautions

⚠ NEVER run malware on your personal device or primary network!

Isolated Lab Environment:

- Use a dedicated physical machine or virtual machine
- Isolate from your main network (air-gapped or separate VLAN)
- Consider using a "throwaway" Google account if needed

Network Isolation:

- Use a dedicated analysis network
- Consider using **InetSim** or **FakeNet** to simulate internet services
- Use firewall rules to prevent accidental C2 callbacks to real victims

GLA UNIVERSITY MATHURA

Types of Sandboxes

Sandboxing is a security practice that isolates programs or files in a virtual environment to safely analyze them without risking the rest of a system.

1. Local Emulator Sandboxes

- **Android Studio Emulator** – Official Google emulator
- **Genymotion** – Commercial, high-performance
- **Use Case:** Basic dynamic analysis, controlled environments

2. Physical Device Sandboxes

- **Rooted Android Phones** – Actual hardware with monitoring tools
- **Use Case:** Bypassing emulator detection, realistic analysis

3. Cloud Sandboxes

- **Any.run** – Interactive, real-time control
- **Joe Sandbox** – Enterprise-grade analysis
- **Use Case:** Quick analysis without local setup

4. Open-Source Sandboxes

- **MobSF** – All-in-one static + dynamic analysis
- **DroidBox** – Behavioral analysis focused
- **Use Case:** Self-hosted, customizable analysis

GLA UNIVERSITY MATHURA

Reverse-Engineering Obfuscated Applications (jadx, Ghidra)

When reverse-engineering obfuscated applications, the techniques and tools used depend on the platform and nature of the obfuscation. JADX is primarily for Android applications, while Ghidra is a multi-purpose tool for analyzing compiled code

Using JADX for obfuscated Android applications: JADX is a decompiler that converts Android DEX bytecode (the format used by Android apps) into human-readable Java source code.

Common obfuscation challenges

- **Renaming:** Obfuscators change meaningful class, method, and variable names to short
- **String encryption:** Important strings, like API keys or URLs, are often encrypted.
- **Control flow obfuscation:** The code's execution path is intentionally complicated with extra jumps and traps to confuse decompilers.

Reverse-Engineering Obfuscated Applications (jadx, Ghidra)

Using Ghidra for obfuscated applications: Ghidra is a powerful reverse-engineering framework from the NSA for analyzing compiled code across many different CPU architectures. It is used when an application contains native, compiled libraries (e.g., C or C++) which JADX cannot process.

Common obfuscation challenges

- **Packed binaries:** The binary may be compressed or encrypted to protect it until runtime. You must unpack it first using dynamic analysis or other techniques.
- **Control flow flattening:** Similar to Java, control flow can be manipulated to make the program's logic hard to follow in Ghidra's disassembly and decompilation views.
- **Indirect calls:** Instead of calling a function directly, the program may use a complex chain of instructions to calculate the function's address at runtime, making static analysis difficult.

RAT for Mobile

A Remote Access Trojan (RAT) is a type of malware that grants an attacker **administrative-level control** over an **infected computer** or **network device.**
Like other trojans, a RAT disguises itself as legitimate software to trick users into downloading and installing it. Once the malware is installed, the attacker can operate the compromised machine covertly and without the user's knowledge

GLA UNIVERSITY MATHURA

How Remote Access Trojans Work

How a RAT works: A RAT infection typically unfolds in three stages:

1. **Infection:** The RAT is delivered to the target's device through social engineering tactics, such as phishing emails, malicious links, or bundled downloads with seemingly harmless software.
2. **Installation and backdoor:** Once executed by the victim, the RAT installs itself and opens a "backdoor" into the system. It often disguises itself as a legitimate file or service to evade detection by the user and security software.
3. **Command and control:** The RAT connects to a command-and-control (C&C) server operated by the attacker. This allows the attacker to issue commands and receive stolen data from the infected device

GLA UNIVERSITY MATHURA

Countermeasures: How to protect against RATs

Protecting against RATs requires a multi-layered security approach:

- **Use security software:** Keep antivirus and anti-malware software updated and run regular system scans.
- **Update and patch:** Ensure your operating system and all software are kept up to date with the latest security patches to fix vulnerabilities.
- **Be careful with downloads:** Practice caution when opening email attachments or downloading files from unknown or untrusted sources.
- **Practice safe browsing:** Avoid clicking on suspicious links or pop-ups.
- **Monitor for signs of infection:** Watch for signs of a RAT infection, such as unusually high network traffic, a slow computer, or unexplained system changes.

Cloud Security

Introduction to cloud computing

Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort or service provider interaction.

It allows users to access and use resources without needing to own or manage the underlying infrastructure physically.

GLA UNIVERSITY MATHURA

Key Characteristics

1. **On-Demand Self-Service**
   - Users can provision computing resources as needed automatically, without human intervention from the service provider.
2. **Broad Network Access**
   - Services are available over the network and accessed through standard mechanisms (e.g., web browsers, mobile apps, APIs).
3. **Resource Pooling**
   - Providers use multi-tenancy to serve multiple customers from a shared pool of resources, dynamically allocated based on demand.
4. **Rapid Scalability**
   - Resources can be scaled up or down according to user needs, appearing unlimited to the consumer.
5. **Measured Service**
   - Resource usage is monitored, controlled, and reported—enabling pay-as-you-go or metered billing models.

GLA UNIVERSITY MATHURA

Cloud Deployment Models

Public Cloud

- Owned and operated by third-party providers, delivering services over the public internet.
- **Example:** AWS, Azure, Google Cloud Platform.
- **Advantages:** Low cost, scalability, maintenance-free.

Private Cloud

- Exclusively used by a single organization; can be hosted internally or externally.
- **Advantages:** Enhanced security, full control, compliance support.

Hybrid Cloud

- Combines public and private clouds, enabling data and application portability.
- **Advantages:** Flexibility, optimized performance, disaster recovery options.

GLA UNIVERSITY MATHURA

Public Cloud Deployment Model

A **Public Cloud** is a cloud computing model in which computing resources (like servers, storage, and applications) are **owned and operated by third-party cloud service providers** and **Key Characteristics:**

Shared Infrastructure: Multiple organizations (tenants) share the same cloud infrastructure.

Pay-as-you-go Model: Users pay only for the resources they consume (e.g., compute hours, storage).

Scalability: Virtually unlimited resources are available on demand.

Accessibility: Services are accessible via the internet from anywhere, anytime.

Maintenance-Free: The cloud provider manages all hardware, software updates, and infrastructure maintenance.

Multi-Tenancy: Resources are dynamically allocated among multiple users securely.

are **made available to the general public** over the internet.

GLA UNIVERSITY MATHURA

Private Cloud Deployment Model

A **Private Cloud** is a cloud computing model where the **cloud infrastructure is exclusively used by a single organization**. It can be **managed internally** or by a **third-party provider**, and it may be **hosted on-premises** or **in a dedicated data center**.

Key Characteristics:

Single-Tenant Environment: All computing resources are dedicated to one organization — no sharing with others.

Greater Control: The organization has full control over infrastructure, data, and security policies.

Customization: Infrastructure and services can be tailored to specific business or regulatory needs.

Enhanced Security: Data and resources remain within the organization's boundaries or under its governance.

Can be On-Premises or Off-Premises: Hosted in the company's data center or by a trusted third-party provider.

GLA UNIVERSITY MATHURA

Hybrid Cloud Deployment Model

A **Hybrid Cloud** is a **combination of public and private cloud environments**, designed to **share data and applications** between them.
It enables organizations to **leverage the scalability and cost-efficiency of the public cloud** while **maintaining control and security of critical workloads** in the private cloud.

Key Characteristics:

Integration of Environments: Combines **private cloud (on-premises)** and **public cloud (off-premises)** infrastructure.

Data and Application Portability: Workloads can move seamlessly between clouds as per business or technical requirements.

Scalable and Flexible: Use private cloud for sensitive operations and public cloud for scaling or high-demand tasks.

Unified Management: Managed through common orchestration tools to monitor and control both environments.

GLA UNIVERSITY MATHURA

Benefits of Cloud Computing

- **Cost Efficiency:** No capital investment in hardware; pay only for what you use.
- **Scalability:** Easily adjust resources based on demand.
- **Accessibility:** Access data and applications from anywhere.
- **Performance:** High-speed computing with minimal downtime.
- **Disaster Recovery & Backup:** Automatic data backup and replication across data centers.
- **Innovation Agility:** Quick deployment of new applications and services.

GLA UNIVERSITY MATHURA

Examples of Cloud Service Providers

Provider	Popular Services
Amazon Web Services (AWS)	EC2, S3, Lambda, RDS
Microsoft Azure	Virtual Machines, Azure SQL, Functions
Google Cloud Platform (GCP)	Compute Engine, Cloud Storage, BigQuery
IBM Cloud	Watson AI, Kubernetes Service
Oracle Cloud	Autonomous Database, OCI Compute

GLA UNIVERSITY MATHURA

Applications of Cloud Computing

Education: Online learning platforms and data management systems.

Healthcare: Electronic health records, telemedicine, and analytics.

Finance: Fraud detection, data security, and scalable transaction systems.

E-commerce: Hosting web apps, managing customer data, and scaling resources.

Research and Development: High-performance computing (HPC) for simulations and modeling.

GLA UNIVERSITY MATHURA

Types of cloud computing

- Infrastructure as a Service (IaaS)
- Platform as a Service (PaaS)
- Software as a Service (SaaS)

GLA UNIVERSITY MATHURA

Infrastructure as a Service (IaaS)

IaaS provides the **basic building blocks of cloud computing**—virtual servers, storage, and networking—on a pay-as-you-go basis.

Users manage **operating systems, applications, and runtime**, while the provider manages the physical infrastructure.

Examples:

- **Amazon Web Services (AWS EC2)**
- **Microsoft Azure Virtual Machines**
- **Google Compute Engine (GCE)**

Use Case:

- Hosting virtual machines and applications
- Data backup and disaster recovery
- Scalable website hosting

Key Benefits:

- High scalability and flexibility
- Cost-effective (no physical hardware needed)
- Full control over OS and applications

GLA UNIVERSITY MATHURA

Platform as a Service (PaaS)

PaaS offers a **ready-to-use development and deployment environment**. Developers can **build, test, and deploy software applications** without managing the underlying servers, storage, or networking.

Examples:

- **Google App Engine**
- **Microsoft Azure App Services**
- **Heroku**

Use Case:

- Web and mobile app development
- Continuous integration and delivery (CI/CD)
- Building APIs and microservices

Key Benefits:

- Simplifies development workflows
- Reduces coding and infrastructure complexity
- Automatic scalability and updates

GLA UNIVERSITY MATHURA

Software as a Service (SaaS)

SaaS delivers **fully functional applications** over the internet. Users simply access the software through a web browser—no installation or maintenance required.

Examples:

- **Google Workspace (Gmail, Docs)**
- **Microsoft 365**
- **Salesforce CRM**
- **Dropbox**

Use Case:

- Email and collaboration tools
- Customer relationship management (CRM)
- Accounting and office productivity

Key Benefits:

- No need for software installation or updates
- Accessible from anywhere, on any device
- Subscription-based cost model

GLA UNIVERSITY MATHURA

Summary Table:

Service Model	User Manages	Provider Manages	Example
IaaS	OS, Applications, Data	Servers, Storage, Networking	AWS EC2, Azure VM
PaaS	Applications, Data	OS, Middleware, Infrastructure	Google App Engine
SaaS	Only Usage	Everything (App + Infra)	Gmail, Salesforce

GLA UNIVERSITY MATHURA

Docker Container Security

Docker containers package applications and dependencies into isolated units. Security ensures these containers cannot be exploited to compromise the system.

Key Security Concepts:

Isolation: Containers should be isolated from each other and the host OS.

Image Security:

- Use **trusted base images** only.
- Regularly **scan images** for vulnerabilities (using tools like *Trivy*, *Clair*, *Anchore*).

Least Privilege:

- Run containers as non-root users.
- Limit capabilities and privileges (e.g., via --cap-drop).

Secrets Management: Avoid hardcoding credentials. Use Docker secrets or environment variables securely.

Network Security:

- Restrict open ports and apply network policies.

Runtime Security:

- Monitor containers for unusual activity.

GLA UNIVERSITY MATHURA

Docker Engine

The **Docker Engine** is the **core component** of the Docker platform. It is responsible for **building, running, and managing containers** on a host system.

Components of Docker Engine:

Docker Daemon (dockerd):

Runs in the background**; manages images, containers, networks, and volumes**.
Listens to Docker API requests.

Docker CLI (Command-Line Interface): The user-facing tool (docker command) used to interact with the Docker Daemon via REST API.

Docker REST API: Provides an interface for external tools or programs to communicate with the Docker Daemon programmatically.

GLA UNIVERSITY MATHURA

Docker Images

A **Docker Image** is a **read-only template** that contains the **application code, libraries, dependencies, tools, and configuration** needed to run a container.

Key Points:

- Built using a **Dockerfile** (a script defining build instructions).
- Immutable — cannot be changed after creation.
- Stored locally or in a remote **Docker Registry**.

docker build -t myapp:1.0

Creates an image named myapp with tag 1.0.

GLA UNIVERSITY MATHURA

Docker Containers

A **Docker Container** is a **runtime instance of a Docker Image**. It includes everything needed to run an application — code, dependencies, and environment.

Characteristics:

- Lightweight and isolated.
- Starts quickly.
- Can be easily created, stopped, moved, or deleted.
- Uses the **image** as a blueprint.

docker run -d -p 8080:80 nginx - This command starts an Nginx web server accessible at **http://localhost:8080**

docker psdocker - Shows all **running containers**

stop <container_id>

Docker Hub

Docker Hub is the **official cloud-based repository** for storing, sharing, and distributing Docker images.

Key Features:

- Hosts **official images** (e.g., Ubuntu, Nginx, MySQL).
- Allows users to **push/pull** custom images.
- Supports **automated builds** and **webhooks** for CI/CD integration.

GLA UNIVERSITY MATHURA

Docker REST API

The **Docker REST API** enables **programmatic access** to the Docker Daemon.

It allows other applications, scripts, or management tools to control Docker remotely.

Common Operations via REST API:

- List images or containers
- Create, start, stop containers
- Manage networks and volumes

GLA UNIVERSITY MATHURA

Docker Registry

A Docker Registry refers to a **storage and distribution system** for **Docker images**. It acts as a central location where you can push (upload) and pull (download) Docker images. Registries can be public, allowing anyone to access and use images, or private, restricting access to authorized users or organizations..

Types:

- **Public Registry:** Docker Hub (default).
- **Private Registry:** Self-hosted (for internal enterprise use).

Functions:

- **Store** built images.
- **Serve** images to Docker clients when pulled.
- **Enable version control** through image tags.

GLA UNIVERSITY MATHURA

Exploiting S3 Buckets

Amazon S3 is a **Simple Storage Service** in AWS that stores files of different types like Photos, Audio, and Videos as Objects providing more scalability and security. It allows the users to store and retrieve any amount of data at any point in time from anywhere on the web. It facilitates features such as extremely high availability, security, and simple connection to other AWS Services.

Common Misconfigurations:

Publicly accessible buckets:

- Buckets set to “public-read” or “public-write”.

Weak permissions:

- Improper IAM (Identity and Access Management) policies or Access Control Lists (ACLs).

Open bucket listings:

- Allowing listing of bucket contents.

GLA UNIVERSITY MATHURA

Cont...

Exploitation Techniques (for ethical testing):

- **Bucket Enumeration:** Finding open buckets with predictable names.
- **Data Extraction:** Reading or downloading exposed files.
- **Privilege Escalation:** Using misconfigured policies to gain write/delete access.

Prevention:

- Use AWS Config or S3 Block Public Access.
- Enforce IAM (Identity and Access Management) least privilege.
- Use server-side encryption (SSE).
- Monitor with AWS CloudTrail.

GLA UNIVERSITY MATHURA

Prevention 1: Use AWS Config or S3 Block Public Access

What they do:

- **S3 Block Public Access** prevents any bucket or object from becoming public—no matter what the IAM policy or ACL says.
- **AWS Config** continuously checks whether your buckets follow security rules (e.g., "no public buckets").

Why it helps:

- It ensures no accidental configuration exposes your data.

How it prevents exploitation:

- Stops public-read/public-write settings.
- Detects policy misconfigurations.
- Alerts you if a bucket becomes risky.

GLA UNIVERSITY MATHURA

Prevention 2: Enforce IAM Least Privilege

What it means:

- Only grant **exact permissions needed**—nothing more.

Example:

- If a user only needs to upload files:
  Use s3:PutObject instead of s3:*.

Why it helps:

- Minimizes damage if:
- A user account is hacked
- An internal user misuses privileges
- A role is misconfigured

How it prevents exploitation:

- Reduces exposure area
- Limits attackers from deleting or manipulating data
- Prevents access to unintended buckets or objects

GLA UNIVERSITY MATHURA

Prevention 3: Use Server-Side Encryption (SSE)

What it does:

- Encrypts your data **at rest** in S3.

Types:

- **SSE-S3** – managed by AWS
- **SSE-KMS** – uses KMS keys for tighter access control
- **SSE-C** – customer-provided keys (less common)

Why it helps:

- Even if someone gains access to S3 storage behind the scenes, the data remains unreadable.

How it prevents exploitation:

- Protects data from being exposed in plaintext
- Restricts access to objects unless IAM + KMS permissions are correct
- Adds an extra layer of security during breaches

GLA UNIVERSITY MATHURA

Prevention 4: Monitor with AWS CloudTrail

What it does:

- Logs every API call made in your AWS account, including:
  - GetObject
  - PutObject
  - ListBucket
  - DeleteObject

Why it helps:

- Allows detection of suspicious activity such as:
- Unexpected object downloads
- Attempts to change bucket permissions
- Large data exfiltration patterns

How it prevents exploitation:

- Helps detect attacks in real time
- Enables alerts through CloudWatch
- Provides audit trails for investigation
- Adds accountability (who did what?)

GLA UNIVERSITY MATHURA

s3scanner

s3scanner is an open-source **Amazon S3 bucket enumeration and auditing tool** used by penetration testers, bug bounty hunters, and cloud security professionals.
It helps identify **publicly accessible or misconfigured S3 buckets** that may expose sensitive data.
Goal: To **find, test, and report insecure S3 buckets** (Simple Storage Service) in a given list of domains or bucket names.

2. Purpose and Use Cases

- **Security Assessment:** Identify S3 buckets that are publicly accessible or misconfigured.
- **Vulnerability Scanning:** Check whether buckets allow *read*, *write*, or *list* permissions.
- **Incident Response:** Detect data exposure or leaks caused by human error.
- **Compliance Auditing:** Verify that cloud storage complies with privacy and data protection standards.

GLA UNIVERSITY MATHURA

s3Inspector

s3Inspector is an **open-source security auditing tool** for **Amazon S3 (Simple Storage Service)**.
It helps identify **misconfigured, publicly accessible, or vulnerable S3 buckets** by testing access permissions and generating detailed reports.
It is primarily used by:

- Cloud security professionals
- Penetration testers
- DevSecOps teams
- Compliance auditors

Objective: To **inspect S3 buckets for security weaknesses** such as public read/write/list access and misconfigured ACLs or IAM policies.

GLA UNIVERSITY MATHURA

Comparison of S3Scanner vs S3Inspector:

Feature	S3Scanner	S3Inspector
Primary Purpose	Discover publicly accessible S3 buckets	Audit and analyze S3 bucket permissions & security
Usage Type	External enumeration	Internal auditing
Input Required	List/wordlist of bucket names	AWS credentials (IAM user/role)
Focus	Finding open buckets	Checking configuration, ACLs, IAM policies
Output	Public-read/write findings	Detailed security report of bucket settings
Typical Use Case	Identify exposed buckets during security tests	Ensure compliance and correct bucket configurations
Risk Level Detected	Public exposure	Misconfigurations, overly permissive policies
Environment	No AWS access required	Requires AWS access
Nature of Tool	Offensive/external security testing	Defensive/internal security auditing

GLA UNIVERSITY MATHURA

Introduction to GRC

Introduction to GRC:

GRC, which stands for **Governance, Risk Management, and Compliance**, is a strategic framework that helps organizations align their operations with business goals, manage risks effectively, and meet regulatory requirements. It integrates these three separate functions into a unified model to improve efficiency, reduce waste, and ensure ethical, legal, and policy adherence across the company. GRC's goal is to enable organizations to achieve principled performance by reliably meeting objectives, addressing uncertainty, and acting with integrity

- A structured approach to aligning IT and business operations with organizational goals, while effectively managing risk and meeting compliance requirements.
- It’s about doing the right thing (Governance), understanding and navigating uncertainty (Risk), and following the rules (Compliance).
- GRC is not just about avoiding fines; it's about creating value, ensuring long-term sustainability, and making informed decisions.
- It's proactive, not just reactive. A strong GRC program doesn't just prevent problems; it identifies opportunities for improvement and efficiency.

GLA UNIVERSITY MATHURA

Components of GRC



GLA UNIVERSITY MATHURA

Components of GRC

Governance

- Set of rules , policies and operations that a company uses to achieve its business goals.

Risk management

- Risk management helps businesses identify risks and find ways to remediate any that are found

Compliance

- Compliance is the act of following rules, laws, and regulations.

Why GRC Matters

- Growing complexity of **regulations**, **cyber threats**, and **business operations**
- Minimizes **financial**, **operational**, and **reputational** risks
- Supports **strategic decision-making**
- Reduces costs through **unified processes**.
- Builds **stakeholder trust** (customers, investors, regulators)

GLA UNIVERSITY MATHURA

Governance

Governance refers to **frameworks, rules, processes**, and **leadership structures** that direct an organization.

Ensures:

- Alignment with **mission and vision**
- Ethical behavior
- Transparent communication
- Accountability at all levels

Key Components of Governance

- Strategic Planning
- Organizational Structure & Roles
- Policies & Procedures
- Performance Measurement & KPIs
- Decision-Making Frameworks
- Internal Controls

GLA UNIVERSITY MATHURA

Objectives of Good Governance

- Improve **organizational effectiveness**
- Ensure **responsibility** and **accountability**
- Facilitate **compliance** with laws and ethics
- Enable **sustainable growth**
- Enhance **transparency** and **trust**

Risk Management

A **systematic process** of identifying, assessing, and controlling risks that may impact organizational objectives.

Involves:

- Identifying threats/opportunities
- Measuring likelihood + impact
- Implementing mitigation measures

Types of Risks

- **Strategic Risks** (market changes, competition)
- **Operational Risks** (process failures, human error)
- **Financial Risks** (liquidity, credit issues)
- **Cyber/Information Security Risks** (breaches, ransomware)
- **Compliance/Legal Risks**
- **Reputational Risks**

GLA UNIVERSITY MATHURA

Risk Management Process

- Risk Identification
- Risk Assessment
- Risk Prioritization
- Risk Mitigation
- Monitoring & Review
- Reporting & Communication

Compliance

Ensuring the organization follows all **laws**, **regulations**, **standards**, and **internal policies** relevant to its operations.

Includes:

- Regulatory compliance
- Industry standards
- Contractual obligations
- Ethical standards & internal policies

Compliance Activities

- Policy creation and enforcement
- Monitoring & audits
- Internal training and awareness
- Reporting violations
- Documentation and evidence collection

GLA UNIVERSITY MATHURA

How GRC Works Together

- **Governance** sets direction, policies, and accountability for IT operations.
- **Risk Management** identifies and mitigates threats that could impact IT assets and services.
- **Compliance** ensures adherence to laws, standards, and internal controls.
- Together, they create a **coherent security posture**, reduce operational gaps, and ensure **regulated, predictable, and resilient IT environments**.
- Integration enables **continuous monitoring**, **faster decision-making**, and **reduced security and regulatory failures**.

GLA UNIVERSITY MATHURA

GRC in an Organization's Information Security Program

1. Governance – Setting Direction & Rules:

Example: A company's leadership creates an **Information Security Policy** that states:

- All employee devices must use strong passwords
- Sensitive data must be encrypted
- Only authorized staff can access customer information

This policy provides **direction, structure, and expectations** for how security should function across the organization.

2. Risk Management – Identifying and Handling Security Threats

Example: The security team identifies a risk:

Employees may fall for phishing emails, leading to data breaches.

They assess the risk and decide to:

- Conduct phishing-awareness training
- Enable email filtering
- Implement multi-factor authentication (MFA)

This shows how the organization **identifies, evaluates, and mitigates risks** to protect its assets.

GLA UNIVERSITY MATHURA

GRC in an Organization's Information Security Program

3. Compliance – Following Laws, Standards, and Policies

Example:

The company must comply with **ISO 27001** requirements.

To do this, they:

- Maintain audit logs
- Protect customer data
- Perform regular security audits
- Document processes and controls

Compliance ensures the organization **meets legal obligations**, industry standards, and its own internal policies.

🎯 **Putting It Together (Integrated Example)**

A company's leaders set security policies (**Governance**), the security team identifies and fixes vulnerabilities (**Risk Management**), and the organization ensures it follows laws and standards like **ISO 27001** (**Compliance**).

Together, this creates a **secure, trustworthy, and well-regulated information security environment**.

GLA UNIVERSITY MATHURA

Importance of GRC in Information Security

- **Aligns security with business goals:** Ensures security initiatives support organizational strategy.
- **Identifies and reduces risks:** Structured risk assessments help prevent data breaches, cyberattacks, and system failures.
- **Ensures regulatory compliance:** Helps meet standards like ISO 27001, NIST, PCI-DSS, GDPR, reducing penalties and legal exposure.
- **Enhances security governance:** Establishes clear policies, accountability, and decision-making structures.
- **Improves incident response:** Integrated controls enable faster detection, reporting, and mitigation of security events.
- **Supports continuous monitoring:** Ensures ongoing evaluation of controls, vulnerabilities, and compliance gaps.
- **Strengthens stakeholder trust:** Demonstrates commitment to protecting information assets.

GLA UNIVERSITY MATHURA

Principles of security audits and assurance

Security Audit:

- **A Specific *Event*.** A formal, point-in-time examination.
- **Focus:** Independently verifying and validating controls against a specific **standard or framework** (e.g., ISO 27001, NIST).
- **Output:** A formal report with findings and evidence.
- **Analogy:** The annual medical check-up with a doctor

Security Assurance:

- **The Ongoing *Process*.** A broad, continuous program of activities.
- **Focus:** Building confidence that security controls are effective *over time*.
- **Includes:** Design reviews, risk assessments, training, monitoring.
- **Analogy:** The daily diet and exercise for good health.

Together, they verify **effectiveness**, **compliance**, and **resilience** of security controls.

GLA UNIVERSITY MATHURA

Why Are Security Audits Essential?

Verify Compliance: Demonstrate adherence to laws, regulations, and contractual obligations (e.g., GDPR, SOX, PCI-DSS).

Identify Vulnerabilities: Uncover weaknesses in systems, processes, and controls *before* they are exploited.

Manage Third-Party Risk: Assess the security of partners and vendors.

Build Trust & Confidence: Provide assurance to customers, investors, and management.

GLA UNIVERSITY MATHURA

Roles and responsibilities in audits

- **Audit Team (Internal/External Auditors):** Conducts independent evaluations of controls, processes, and compliance to identify gaps and risks.
- **Management:** Provides resources, supports the audit process, and ensures remediation of audit findings.
- **IT & Security Teams:** Supplies technical evidence, demonstrates control implementation, and assists in resolving identified issues.
- **Process Owners:** Explain business workflows, provide process-related documentation, and maintain effective operational controls.
- **Risk & Compliance Teams:** Interpret requirements, map controls to regulations, and track remediation to maintain compliance posture.
- **Executive Leadership / Audit Committee:** Oversees audit activities, reviews major findings, and ensures accountability and resource allocation.

GLA UNIVERSITY MATHURA

Introduction to audit frameworks,

Audit frameworks are **structured, standardized methodologies** that guide how audits are planned, conducted, and evaluated. They provide a **consistent, repeatable, and objective approach** to assessing an organization's controls, policies, risks, and compliance posture.

Audit frameworks are essential because they:

- Ensure **uniformity and reliability** in audit results
- Align audits with **industry best practices and regulatory requirements**
- Improve **accuracy, transparency, and accountability**
- Support organizations in identifying gaps and strengthening their security posture
- Provide **benchmark criteria** for certifications and compliance assessments

Common audit frameworks include **ISO 27001**, **NIST Cybersecurity Framework**, **COBIT**, **SP 800-53**

GLA UNIVERSITY MATHURA

ISO 27001

ISO 27001 is an international standard that specifies the requirements for an **Information Security Management System (ISMS).** It provides a framework for organizations to **manage, monitor, and improve** the security of their information assets by ensuring the **confidentiality**, **integrity**, and **availability** of information..

It is a globally recognized standard that provides a **framework for**

- **Building**
- **Implementing**
- **Maintaining,**
- **Improving**

an organization's information security management system.

GLA UNIVERSITY MATHURA

Key Points About ISO 27001

Purpose: Protect confidentiality, integrity, and availability of information.

Scope: Applies to people, processes, technology, and physical security.

Risk-Based: Requires organizations to identify and treat security risks systematically.

Mandatory Documents: Policies, procedures, risk assessment reports, control implementation evidence.

Annex A Controls: Contains **93 security controls** categorized into themes such as access control, physical security, operations security, encryption, supplier management, and incident management.

Certification: Organizations can get certified through accredited auditors to prove their security maturity.

GLA UNIVERSITY MATHURA

Why ISO 27001 is Important

- Ensures **structured, standardized** security practices
- Builds **customer trust**
- Helps meet **legal and regulatory requirements**
- Reduces risk of breaches and cyber incidents
- Supports continuous improvement of security posture

GLA UNIVERSITY MATHURA

NIST SP 800-53

NIST SP 800-53 stands for **National Institute of Standards and Technology Special Publication 800-53**.

It provides a **comprehensive catalog of security and privacy controls** designed to protect **U.S. federal information systems**, but it is widely used by private organizations globally to strengthen cybersecurity programs.

Security and privacy controls are the **safeguards, rules, and mechanisms** an organization puts in place to **protect information, systems, and user data** from threats, misuse, and unauthorized access.

GLA UNIVERSITY MATHURA

NIST SP 800-53 - Key Points

Purpose: To provide a standardized set of security and privacy controls to ensure confidentiality, integrity, and availability of information systems.

Scope: Applies to federal agencies, contractors, and organizations seeking strong security baselines.

Control Families: Controls are organized into categories such as Access Control, Audit & Accountability, Incident Response, Risk Assessment, System & Communications Protection, etc.

Risk-Based: Supports selecting and tailoring controls based on risk and system impact level (**Low, Moderate, High**).

GLA UNIVERSITY MATHURA

COBIT

COBIT stands for **Control Objectives for Information and Related Technologies**. It is a globally recognized **IT governance and management framework** developed by **ISACA** that helps organizations ensure that IT processes align with business goals, manage risks effectively, and meet regulatory requirements.

COBIT is a framework for IT governance and management that focuses on aligning IT with business goals and **managing the entire IT landscape, rather than just security**.

An **IT governance and management framework** is a structured set of **policies, processes, roles, and best practices** that help an organization ensure its IT systems support **business goals, operate efficiently, manage risks, and comply with regulations**.

GLA UNIVERSITY MATHURA

COBIT - Key Points

Purpose: To provide a structured framework for governing and managing enterprise IT.

Scope: Covers security, risk, compliance, performance, value delivery, and IT operations.

Domains: COBIT 2019 organizes practices into focus areas such as **Governance, Risk Management, Information Security, Compliance**, and Performance Management.

Principles-Based: Uses governance and management objectives to help organizations build a holistic IT governance system.

GLA UNIVERSITY MATHURA

Implementing GRC policies

- Define governance structure and policies
- Conduct risk assessments and maintain a risk register
- Map and implement compliance requirements
- Document procedures and assign responsibilities
- Train employees and enforce controls
- Continuously monitor, audit, and improve

GLA UNIVERSITY MATHURA

GRC Policy Implementation for ISO 27001

1. Governance Implementation: Governance ensures **direction, accountability, and structure** for the ISMS.

Develop core policies:

- **Information Security Policy**
- **Access Control Policy**
- **Acceptable Use Policy**
- **Asset Management Policy**

2. Risk Management Implementation: Risk Management ensures the ISMS is **risk-based**, as required by ISO 27001.

- Conduct a **formal risk assessment** using qualitative or semi-quantitative methods.
- Identify threats, vulnerabilities, and impacts on information assets.
- Maintain a **Risk Register** listing risks, owners, controls, and treatment plans.
- Choose controls from **Annex A (93 controls)** based on risk results.
- Document **risk treatment plans** and residual risk acceptance.

3. Compliance Implementation: Compliance ensures the organization **meets ISO 27001 requirements** and any legal or regulatory obligations.

- Map existing controls to ISO 27001 Annex A requirements.
- Implement **procedures and SOPs** that support the policies.
- Maintain evidence: logs, reports, audit trails, meeting minutes, training records.
- Perform **internal ISMS audits** to identify gaps before the external audit.
- Conduct **management review meetings** and track corrective actions (CAPA).

GLA UNIVERSITY MATHURA

GRC Essentials for ISO 27001 Audit Readiness

Policies to Implement Before the Audit

- **Information Security Policy (Governance):** Defines security objectives, responsibilities, and accountability across the organization.
- **Risk Management Policy & Risk Register (Risk Management):** Outlines how risks will be identified, assessed, treated, and monitored as required by ISO 27001.
- **Compliance & Documentation Management Policy (Compliance):** Ensures all ISO 27001 controls, evidence, procedures, and records are properly maintained and audit-ready.

Importance of GRC in Preparing for an Audit

A strong **Governance, Risk Management, and Compliance (GRC)** framework is essential for audit readiness because it provides the **structure, controls, documentation, and accountability** an auditor expects. Effective GRC:

1. Ensures Clear Governance: Establishes **policies, roles, and responsibilities**, helping auditors see who owns which processes.

2. Strengthens Risk Management: Provides documented **risk assessments, risk registers, and treatment plans**, showing the organization understands and manages its security risks.

3. Improves Compliance: Ensures the organization **follows required laws, standards, and internal policies**.

4. Enhances Documentation & Control Evidence: GRC frameworks ensure **procedures, logs, records, and reports** are properly maintained and easily accessible for auditors.

5. Supports Continuous Monitoring: Helps track control performance, incidents, corrective actions, and improvements.

6. Provides Accountability and Transparency: Clarifies who performs, approves, and reviews security tasks.

GLA UNIVERSITY MATHURA

Case Study: Adoption of GRC & ISO 27001 at GLA University IT environment

GLA University manages large volumes of sensitive information, including student records, staff data, financial transactions, research documents, and digital learning systems.

With increasing cyber threats and the shift toward online platforms (ERP, LMS, online admissions), the university identified the need for a **standardized security framework**.

Problem

Before adopting GRC and ISO 27001, GLA University faced several challenges:

- No **centralized governance** for IT and security decisions
- Inconsistent **policies and procedures** across departments
- Risks like unauthorized access, data leakage, and phishing attempts
- Compliance gaps in data protection and documentation
- Difficulty demonstrating security readiness during external audits and accreditation visits

These gaps highlighted the need for a **formal ISMS (Information Security Management System)** based on **ISO 27001**.

Cont…

Framework Adoption Approach

1. Implementing Governance (G)

The University created an **Information Security Steering Committee** involving IT, Academics, Finance, and Administration.

Developed core security policies:

- Information Security Policy
- Data Protection Policy
- Access Control Policy
- Email & Internet Use Policy

Defined roles such as **CISO**, **IT Security Officer**, and **Data Owners**.

Impact: Clear accountability and structured decision-making for security-related activities.

Cont...

2. Implementing Risk Management (R)

Conducted a university-wide **risk assessment** covering ERP, LMS, Wi-Fi networks, research data, and examination systems.

Identified key risks:

- Weak passwords by students/staff
- Unauthorized access to ERP
- Data loss due to system failures
- Malware infections in labs

Created a **Risk Register** and applied treatment actions such as MFA, encryption, network segmentation, and awareness training.

Impact: Prioritized risk areas and implemented appropriate ISO 27001 Annex A controls.

Cont...

3. Implementing Compliance (C)

Mapped controls to ISO 27001 requirements (e.g., access control, asset management, physical security).

Maintained documentation such as:

- SOPs for data access
- Incident management procedures
- Backup and recovery logs
- Asset lists and user access reviews

Conducted internal audits and fixed non-conformities.

Impact: Improved compliance posture and readiness for accreditation and external audits.

Cont…

ISO 27001 ISMS Implementation Steps

GLA University followed a structured approach:

- **Define ISMS scope** (ERP, LMS, data centers, examination systems).
- **Conduct risk assessment** and select Annex A controls.
- **Implement technical controls** – MFA, firewalls, backups, antivirus, encryption.
- **Implement administrative controls** – training, policies, procedures.
- **Internal audit** and management review meetings.
- **Corrective actions** for non-conformities.
- **Prepare for external ISO 27001 audit.**

Cont...

Outcomes

Security Improvements

- Stronger access control across ERP and LMS
- Fewer incidents of phishing and malware due to training
- Better documentation and control evidence for audits

Operational Improvements

- Clear governance and responsibilities
- Streamlined processes across departments
- Faster response to incidents and vulnerabilities

Compliance Benefits

- Alignment with ISO 27001:2022
- Increased trust among students, staff, and regulatory bodies
- Preparedness for NAAC, NBA, and external accreditation audits

Risk Identification Techniques

a) Brainstorming

- Team members discuss possible risks based on experience and knowledge.
- Useful for generating a broad list of risks quickly.

b) Interviews & Questionnaires

- One-on-one discussions with process owners, IT teams, or managers.
- Helps uncover hidden or process-specific risks.

c) Checklists

- Use **predefined lists** of common risks or control requirements (e.g., ISO 27001 Annex A, NIST controls).
- Ensures no major areas are missed.

d) Asset Inventory & Threat Analysis

- Identify **critical assets** (data, systems, people) and examine what threats apply to each.
- Used heavily in ISO 27001.

e) Past Incident Reviews

Analyze previous breaches, outages, and audit findings to identify recurring risks.

GLA UNIVERSITY MATHURA

Risk Assessment Techniques

These methods help evaluate the **likelihood** and **impact** of identified risks to determine their severity and priority.

a) Qualitative Risk Assessment

- Uses descriptive values like *Low / Medium / High*.
- Easy to understand and commonly used in ISO 27001.

b) Quantitative Risk Assessment

- Assigns **numeric values** (e.g., financial loss, probability percentages).
- Useful for calculating risk in measurable terms.

c) Semi-Quantitative Assessment

- Combines qualitative categories with scoring (e.g., 1–5 scale).
- Balances simplicity and precision.

d) Risk Matrix (Probability–Impact Matrix)

- Plots risks on a grid to visualize severity.
- Helps prioritize which risks need immediate action.
- High Probability + High Impact → Critical Risk (immediate action needed)
- Low Probability + Medium Impact → Moderate Risk (monitor)

GLA UNIVERSITY MATHURA

Qualitative vs. Quantitative Risk Analysis

Aspect	Qualitative Analysis	Quantitative Analysis
Basis	Judgment, experience	Numerical data, statistics
Output	High/Med/Low	₹ value, probability
Time Required	Fast	Slow
Accuracy	Moderate	High
Best Used For	General security risks, ISO 27001	Financial impact modeling
Data Required	Low	High
Decision Support	Qualitative prioritization	Financial justification
Complexity	Simple	Complex

GLA UNIVERSITY MATHURA

Risk Mitigation Strategies

Risk mitigation is about selecting appropriate actions to reduce risk to an acceptable level.

1. **Risk Avoidance:** Eliminating the activity that causes the risk.
   Example: Discontinue using an outdated software system.
2. **Risk Reduction (Mitigation):** Implementing controls to reduce likelihood or impact.
   Example: Install firewalls, apply patches, enable MFA, encrypt data.
3. **Risk Transfer:** Shifting the financial impact to a third party.
   Example: Cyber insurance, outsourcing services to secure vendors.
4. **Risk Acceptance:** Accepting the risk because it is within risk appetite or cost of control is too high.
   Example: Accepting minor website downtime risk.
5. **Risk Sharing:** Distributing risk between multiple parties.
   Example: Cloud provider and organization share responsibility for data security.

GLA UNIVERSITY MATHURA